Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА №2

Разработка API
с использованием фреймворка Sinatra

по дисциплине «Методики разработки современных веб-приложений»

Выполнил: студент гр. 5140903/50401
К.И. Забродин

Руководитель: доцент ВШ ПИ
А.В. Иванищев

Санкт-Петербург
2026

1 Цель и задание работы

Цель работы — изучить маршрутизацию Sinatra и подготовить основу API для сервиса рекрутинга.

Необходимо установить зависимости, создать приложение в классическом и модульном стилях, проверить прямой запуск и запуск через Rack. Требуется реализовать именованные параметры, маски splat, регулярное выражение, перенаправление и необязательную часть маршрута, а также подключить пространство имён. В завершение нужно подготовить структуру приложения Sinatra для дальнейшего подключения PostgreSQL.

Работа выполнена в каталоге `sinatra_api` репозитория `github.com/kiriklzz/ruby_api_labs`. Использованы Ruby 3.2.2, Sinatra 4.2.1, Rack и Puma 8.0.2. В методичке приведены более ранние версии Sinatra и серверы Thin/WEBrick; в работе те же способы запуска проверены с Puma.

2 Установка и структура проекта

В `Gemfile` указаны Sinatra, расширение `sinatra-contrib`, Rackup и Puma. Для следующих этапов API подключены драйвер `pg`, ORM Sequel, `sequel-seed`, Rake и Dotenv. Версии зависимостей сохранены в `Gemfile.lock`.

```
cd sinatra_api
bundle install
```

Сохранены отдельные файлы: `classic.rb` для классического стиля и `my_sinatra.rb` для модульного. Конфигурации Rack находятся в `classic.ru` и `modular.ru`. Итоговое приложение собирается через `config.ru`.

```
sinatra_api/
  config/          # настройки подключения
  helpers/         # сериализация и общие методы
  models/          # модели Sequel
  routes/          # маршруты по ресурсам
  lib/tasks/       # задачи Rake
  db/migrations/   # миграции
  seeds/           # исходные данные
  test/            # проверки маршрутов и API
```

Маршруты сгруппированы по ресурсам в `routes`; отдельные контроллеры для этой структуры не потребовались. Миграции, наполнение базы и конечные точки с фильтрами относятся к следующим работам; здесь проверяется основа приложения и загрузка файлов.

3 Классический и модульный стили

3.1 Классический вариант

В `classic.rb` подключён `sinatra`. Маршруты определены на верхнем уровне. Корневой маршрут возвращает приветствие, `/hello/:name` использует имя из URL, а общий `/*` обрабатывает остальные пути.

```
require 'sinatra'
get('/') { 'Hello My Sinatra - Easy and Wide world!' }
get('/hello/:name') do
  "Sinatra приветствует тебя, #{params[:name]}!"
end
```

Прямой запуск выполняется командой `bundle exec ruby classic.rb`. Для Rack создан `classic.ru`:

```
require_relative 'classic'
run Sinatra::Application
```

Проверены оба варианта. На рисунке 1 приведён протокол прямого запуска и реальные ответы. Для параллельной проверки использован порт 4569; стандартный порт Sinatra — 4567.

Классический стиль: запуск и запросы

Лабораторная работа №2 · Протокол фактических запусков и HTTP-ответов · 09.11.2026

```
$ PORT=4569 bundle exec ruby classic.rb -p 4569
== Sinatra (v4.2.1) has taken the stage on 4569 for development with backup
from Puma
Puma starting in single mode...
* Puma version: 8.0.2 ("Into the Arena")
* Ruby version: ruby 3.2.2 (2023-03-30 revision e51014f9c0) [arm64-darwin24]
*  Min threads: 0


GET http://localhost:4569/
HTTP 200 | text/plain;charset=utf-8

Hello My Sinatra - Easy and Wide world!

GET http://localhost:4569/hello/Peter
HTTP 200 | text/plain;charset=utf-8

Sinatra приветствует тебя, Peter!
```

Рис. 1: Прямой запуск классического приложения и HTTP-ответы

3.2 Модульный вариант и запуск через Rack

В `my_sinatra.rb` создан класс `MyAppModule::App`, унаследованный от `Sinatra::Base`. Расширение пространства имён зарегистрировано в классе приложения.

```
require 'sinatra/base'
require 'sinatra/namespace'
module MyAppModule
  class App < Sinatra::Base
    register Sinatra::Namespace
    # определения маршрутов
    run! if app_file == $PROGRAM_NAME
  end
end
```

Файл `modular.ru` подключает класс и передаёт его Rack:

```
require_relative 'my_sinatra'
run MyAppModule::App
```

Выполнены запуск через Ruby-файл и запуск через Rack:

```
bundle exec ruby my_sinatra.rb
bundle exec rackup -p 4567 modular.ru
```

Это альтернативные команды: предыдущий сервер нужно остановить либо выбрать другой порт. При контрольном запуске модульного Ruby-файла порт задан через `PORT=4571`. Для Rack порт задаётся аргументом `-p`.

Для проверки модульного приложения запросы отправлялись HTTP-клиентом. На рисунках приведены URL, статус, тип содержимого и тело ответа. Исходные результаты сохранены в `evidence/requests.json`; интерфейс Postman при проверке не использовался.

3.3 Порядок маршрутов

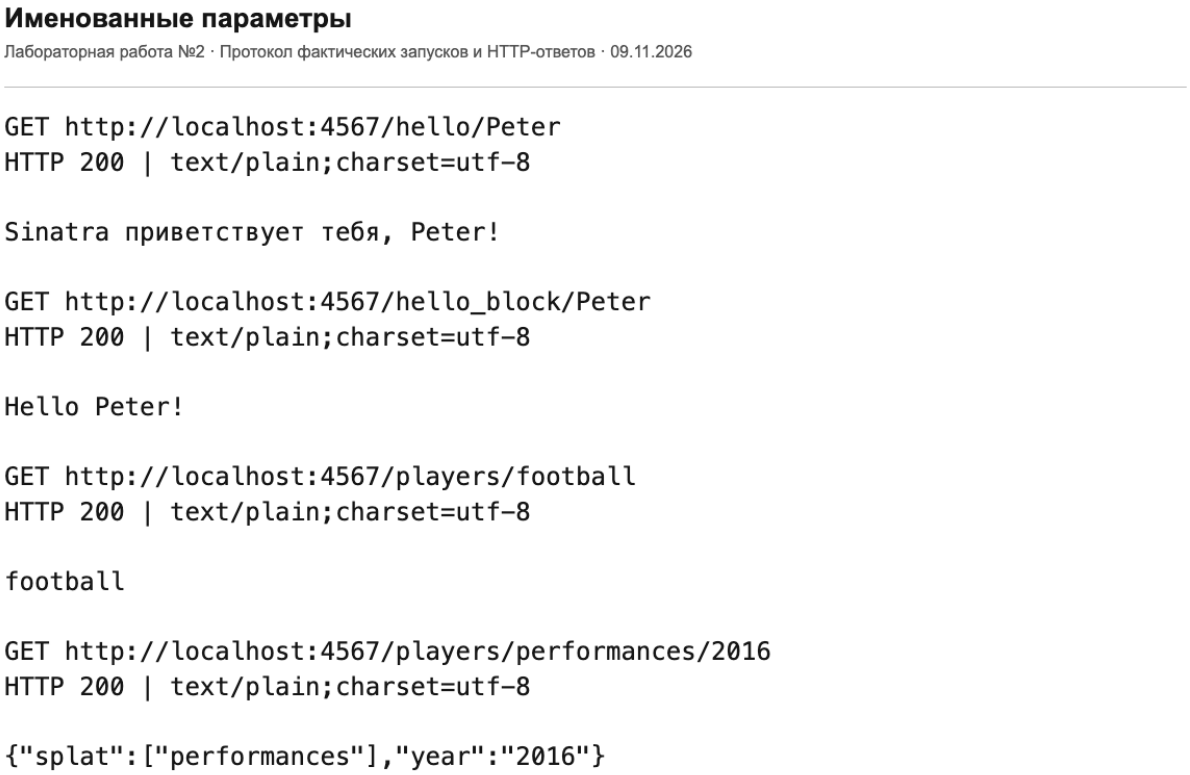
Специальные маршруты объявлены до общего `/*`. Иначе общий обработчик перехватывал бы запросы к приветствию и другим примерам. Маска внутри namespace расположена перед внешней общей маской.

Текстовые примеры возвращаются с типом `text/plain; charset=utf-8`. Поэтому имя отображается как текст и не интерпретируется браузером как HTML.

4 Проверка маршрутов

4.1 Именованные параметры

Запрос `/hello/Peter` получает имя через `params[:name]`. Маршрут `/hello_block/:name` передаёт параметр аргументом Ruby-блока. В `/players/:sport_id` извлекается идентификатор спорта. Маршрут `/players/*/:year` сочетает splat с параметром года.



Именованные параметры

Лабораторная работа №2 · Протокол фактических запусков и HTTP-ответов · 09.11.2026

```
GET http://localhost:4567/hello/Peter
HTTP 200 | text/plain;charset=utf-8

Sinatra приветствует тебя, Peter!

GET http://localhost:4567/hello_block/Peter
HTTP 200 | text/plain;charset=utf-8

Hello Peter!

GET http://localhost:4567/players/football
HTTP 200 | text/plain;charset=utf-8

football

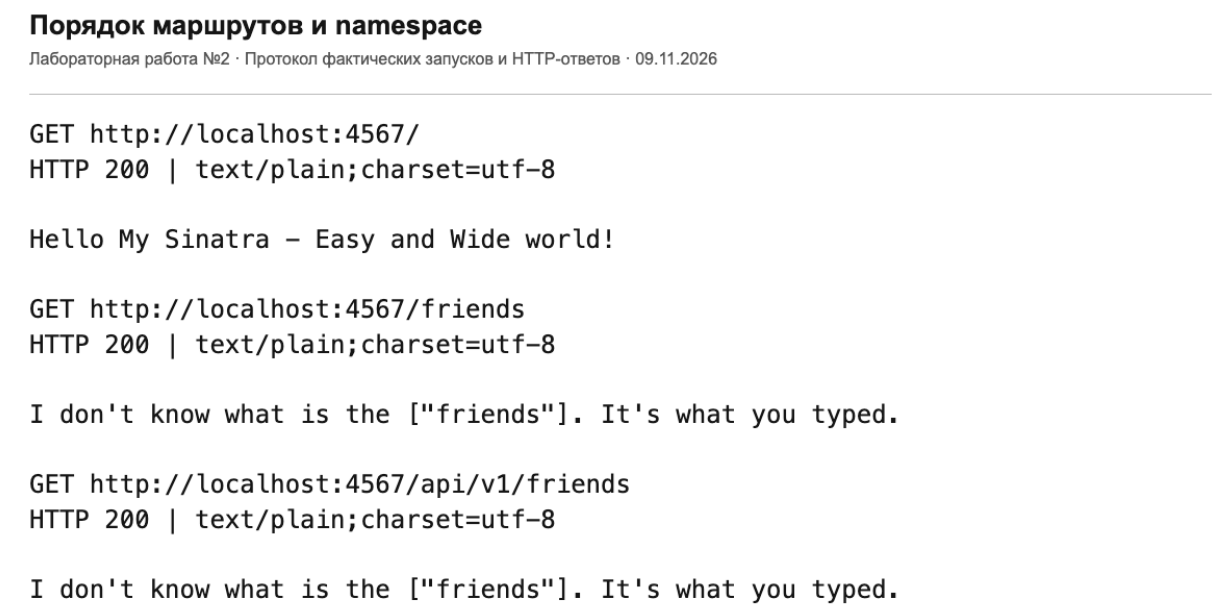
GET http://localhost:4567/players/performances/2016
HTTP 200 | text/plain;charset=utf-8

{"splat":["performances"],"year":"2016"}
```

Рис. 2: Именованные параметры и сочетание с маской

4.2 Namespace

Внутри `namespace '/api/v1'` определён маршрут `/*`. У запроса `/api/v1/friends` параметр splat содержит только `friends`. Запрос `/friends` обрабатывается внешней маской. Корневой маршрут возвращает своё приветствие.



Порядок маршрутов и namespace

Лабораторная работа №2 · Протокол фактических запусков и HTTP-ответов · 09.11.2026

```
GET http://localhost:4567/
HTTP 200 | text/plain;charset=utf-8

Hello My Sinatra - Easy and Wide world!

GET http://localhost:4567/friends
HTTP 200 | text/plain;charset=utf-8

I don't know what is the ["friends"]. It's what you typed.

GET http://localhost:4567/api/v1/friends
HTTP 200 | text/plain;charset=utf-8

I don't know what is the ["friends"]. It's what you typed.
```

Рис. 3: Корневой маршрут и маски внутри и вне namespace

4.3 Перенаправление

Маршрут `/redirect` выполняет `redirect to('/hello/World')`. Автоматический тест отдельно проверяет наличие перенаправления, затем следует по адресу назначения. HTTP-клиент протокола также следует за перенаправлением; на рисунке показан конечный URL и итоговый ответ HTTP 200.

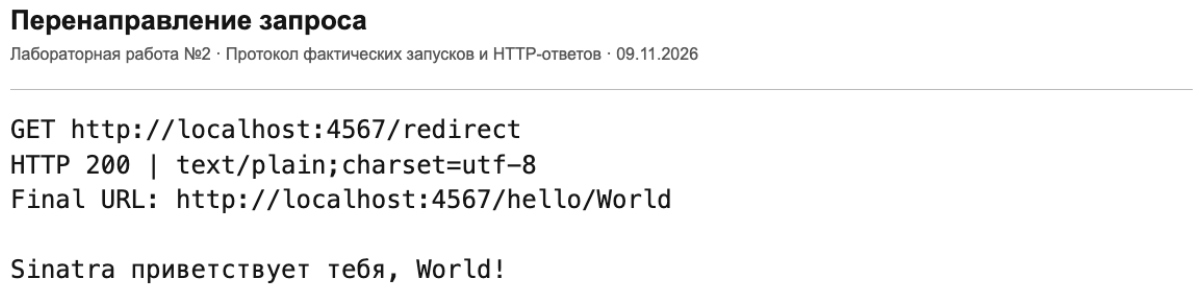


Рис. 4: Переход на маршрут приветствия после перенаправления

4.4 Регулярное выражение и splat

Регулярное выражение извлекает слово после `/meta/hello/`. Запрос `/meta/hello/12345/world` возвращает `Hello, 12345!`. В `/say/*/to/*` извлекаются две строки, а в `/download/*.*` — путь и расширение файла. Массивы представлены JSON-строкой.



Рис. 5: Регулярное выражение, две маски и расширение файла

4.5 Необязательный формат

Шаблон `/jobs.?:format?` обслуживает запросы `/jobs`, `/jobs.json` и `/jobs.xml`. Во всех случаях возвращается одна демонстрационная строка. Суффиксы показывают работу шаблона и не переключают сериализацию ответа.

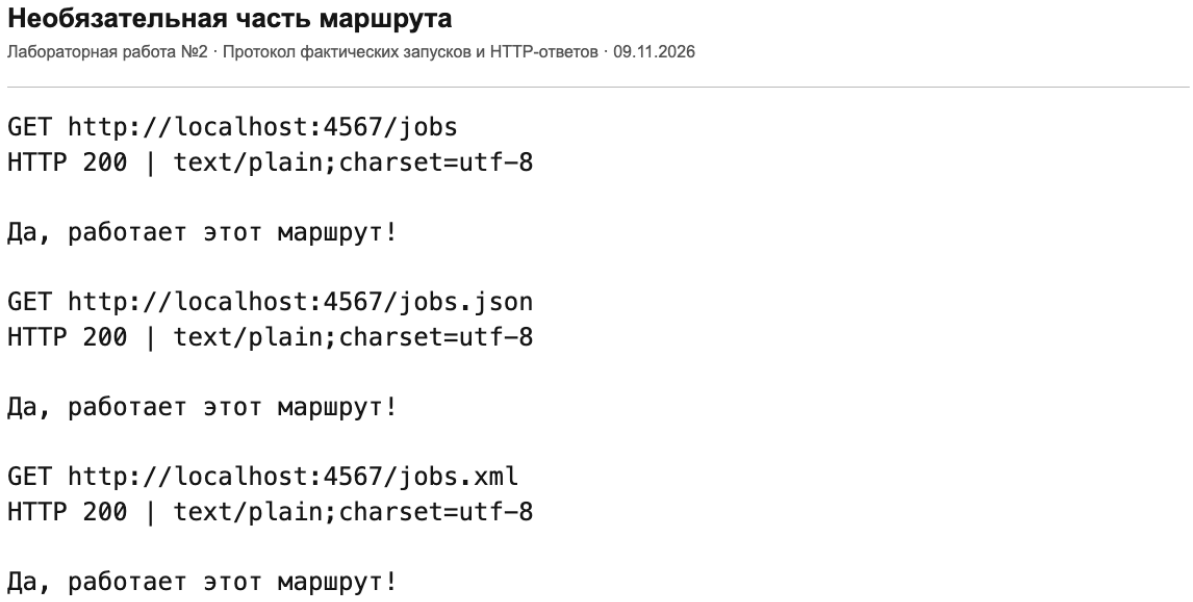


Рис. 6: Запросы с отсутствующим и заданным суффиксом формата

5 Подготовка итогового API

В `api_sinatra_postgres.rb` используется классический стиль. После подключения конфигурации базы загружаются файлы из `models`, `helpers` и `routes`. Пути строятся относительно `__dir__`, поэтому загрузка не зависит от рабочей папки терминала.

Файл `config.ru` запускает `Sinatra::Application`. Итоговое приложение проверено на порту 4568 одновременно с демонстрацией на 4567. Корневой запрос вернул `Hi! I am api_sinatra_postgres!` со статусом HTTP 200.

Итоговая структура: классическое приложение

Лабораторная работа №2 · Протокол фактических запусков и HTTP-ответов · 09.11.2026

```
GET http://localhost:4568/
HTTP 200 | text/html;charset=utf-8

Hi! I am api_sinatra_postgres!

$ cat config.ru
require_relative 'api_sinatra_postgres'
run Sinatra::Application

$ cat api_sinatra_postgres.rb
require 'sinatra'
require 'sinatra/namespace'
require 'json'
require_relative 'config/database'

%w[models helpers routes].each do |dir|
  Dir[File.join(__dir__, dir, '*.rb')].sort.each { |file| require file }
end

get('/') { 'Hi! I am api_sinatra_postgres!' }
```

Рис. 7: Корневой ответ и файлы сборки итогового приложения

6 Автоматическая проверка и результаты

Тесты части 2 находятся в `test/routes_test.rb` и используют Minitest и Rack::Test. Они проверяют параметры, перенаправление, текстовый тип ответа, маски, регулярное выражение, необязательный формат и namespace.

```
bundle exec ruby -Itest test/routes_test.rb
```

Результат отдельного запуска: 3 теста, 18 утверждений, 0 ошибок и 0 неудачных проверок. Прямые запуски и конфигурации Rack дополнительно проверены реальными HTTP-запросами. Общий набор частей 2–4 также прошёл: 10 тестов и 83 утверждения.

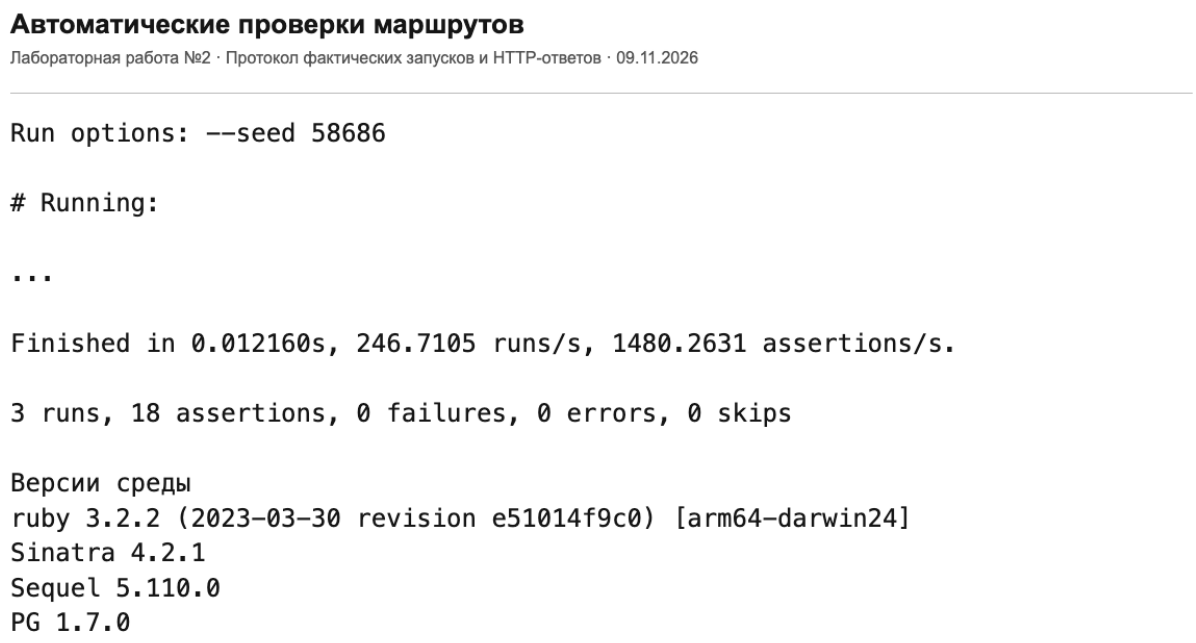


Рис. 8: Результат тестов маршрутизации и версии среды

Приложение запускалось из терминала командами Ruby/Rack. Настройка Run Configuration в RubyMine описана в README проекта, но запуск через эту IDE не проверялся.

7 Вывод

Оба варианта Sinatra-приложения работают при прямом запуске и через Rack. Маршруты извлекают параметры URL, обрабатывают маски и регулярное выражение, выполняют перенаправление и учитывают namespace. Для продолжения работы подготовлена структура API с загрузкой моделей, общих методов и маршрутов.